

Reset answer

```
1  /*
2  * Complete the 'reverseArray' function below.
3  *
4  * The function is expected to return an INTEGER_ARRAY.
5  * The function accepts INTEGER_ARRAY arr as parameter.
6  */
7
8  /*
9  * To return the integer array from the function, you should:
10 *   - Store the size of the array to be returned in the result_count variable
11 *   - Allocate the array statically or dynamically
12 *
13 * For example,
14 * int* return_integer_array_using_static_allocation(int* result_count) {
15 *     *result_count = 5;
16 *
17 *     static int a[5] = {1, 2, 3, 4, 5};
18 *
19 *     return a;
20 * }
21 *
22 * int* return_integer_array_using_dynamic_allocation(int* result_count) {
23 *     *result_count = 5;
24 *
25 *     int *a = malloc(5 * sizeof(int));
26 *
27 *     for (int i = 0; i < 5; i++) {
28 *         *(a + i) = i + 1;
29 *     }
30 *
31 *     return a;
32 * }
33 *
34 */
35 int* reverseArray(int n, int *a, int *nC) {
36     *nC=n;
37     int* b=(int*)malloc(sizeof(int)*n);
38     for(int i=0;i<n;i++)
39     {
40         b[i]=a[n-i-1];
41     }
42     return b;
43 }
44
45
```

|   | Test                                                                                                                                                                                          | Expected              | Got                   |   |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|-----------------------|---|
| ✓ | <pre>int arr[] = {1, 3, 2, 4, 5}; int result_count; int* result = reverseArray(5, arr, &amp;result_count); for (int i = 0; i &lt; result_count; i++)     printf("%d\n", *(result + i));</pre> | 5<br>4<br>2<br>3<br>1 | 5<br>4<br>2<br>3<br>1 | ✓ |

Passed all tests! ✓

```

1  /*
2  * Complete the 'cutThemAll' function below.
3  *
4  * The function is expected to return a STRING.
5  * The function accepts following parameters:
6  * 1. LONG_INTEGER_ARRAY lengths
7  * 2. LONG_INTEGER minLength
8  */
9
10 /*
11 * To return the string from the function, you should either do static allocation or dynamic allocation
12 *
13 * For example,
14 * char* return_string_using_static_allocation() {
15 *     static char s[] = "static allocation of string";
16 *
17 *     return s;
18 * }
19 *
20 * char* return_string_using_dynamic_allocation() {
21 *     char* s = malloc(100 * sizeof(char));
22 *
23 *     s = "dynamic allocation of string";
24 *
25 *     return s;
26 * }
27 */
28
29 #include<stdio.h>
30 int cmp(const void*a,const void*b)
31 {
32     return(*(int*)a)-(*(int*)b);
33 }
34 char* cutThemAll(int n,long *a,long mL)
35 {
36     int s=0;
37     for(int i=0;i<n;i++)
38     {
39         s+=a[i];
40     }
41     long r=s;
42     qsort(a,n,sizeof(long),cmp);
43     for(int i=0;i<n;i++)
44     {
45         if(r==mL)
46         {
47             return "Possible";
48         }
49         if(r>mL)
50         {
51             r-=a[i];
52         }
53     }
54 }

```

|   | Test                                                                      | Expected   | Got        |   |
|---|---------------------------------------------------------------------------|------------|------------|---|
| ✓ | long lengths[] = {3, 5, 4, 3};<br>printf("%s", cutThemAll(4, lengths, 9)) | Possible   | Possible   | ✓ |
| ✓ | long lengths[] = {5, 6, 2};<br>printf("%s", cutThemAll(3, lengths, 12))   | Impossible | Impossible | ✓ |

Passed all tests! ✓